BULACAN STATE UNIVERSITY

College of Information and Communications Technology

Project Documentation

Campus Equipment Borrowing and Inventory System

Technical design, implementation, evaluation and submission evidence

Submission detail	Value
Course / academic year	IT 401W - Web Programming and Frameworks | 2026-2027
Group	Group 1
Section and proponents	To be completed with the verified section, roster and student identifiers before submission.
Prepared snapshot	25 October 2026 | Python 3.13.7 | Django 6.1.2
Submitted to	Dr. Aaron Paul M. Dela Rosa, as named in the official project specification.

Evidence-qualified edition. This document describes the copied implementation and tests executed for run IT401W-G1-20261025. Unexecuted evaluations and unavailable academic records are explicitly identified.

Contents

Chapter 1 - Introduction 5

1.1 Project overview . . . 5

1.2 Background and problem statement . . . 5

1.3 Objectives . . . 5

1.4 Scope, significance and limitations . . . 6

1.5 Options and recommendation . . . 6

Chapter 2 - System Requirements 7

2.1 Source and traceability conventions . . . 7

2.2 Roles and permissions . . . 8

2.3 Business rules . . . 8

2.4 Non-functional and interface requirements . . . 9

Chapter 3 - System Design 10

3.1 Django MTV architecture . . . 10

3.2 Technology stack and module boundaries . . . 11

3.3 Authentication and authorization design . . . 11

3.4 Inventory, reservation and loan design . . . 11

3.5 Maintenance, audit and reporting design . . . 12

Chapter 4 - Database Design 13

4.1 Actual schema and conceptual mapping . . . 13

4.2 Rendered entity relationship diagrams . . . 13

4.3 Relationship register . . . 16

4.4 Data dictionary conventions . . . 17

4.5.1 accounts.User . . . 18

4.5.2 inventory.Category . . . 19

4.5.3 inventory.Equipment . . . 20
4.5.4 inventory.EquipmentUnit . . . 21
4.5.5 borrowing.BorrowingRequest . . . 22
4.5.6 borrowing.BorrowingRequestItem . . . 23
4.5.7 borrowing.Allocation . . . 24
4.5.8 loans.Loan . . . 25
4.5.9 loans.LoanUnit . . . 26
4.5.10 maintenance.MaintenanceRecord . . . 27
4.5.11 dashboard.SystemSetting . . . 28
4.5.12 dashboard.AuditLog . . . 29
4.6 Infrastructure, migrations and validation layers . . . 30

Chapter 5 - System Workflows 31

5.1 Registration, sign-in and browsing . . . 31
5.2 Submission, review and allocation . . . 32
5.3 Release, returns and overdue monitoring . . . 33
5.4 Maintenance and equipment archiving . . . 34

Chapter 6 - System Implementation 35

6.1 Project structure and reproducible setup . . . 35
6.2 Traceable implementation entry points . . . 36
6.3 Transaction and concurrency strategy . . . 36
6.4 Maintenance, backup and operational guidance . . . 36
6.5 Technology evaluation: optional barcode scanning . . . 37

Chapter 7 - Testing and Evaluation 38

7.1 Method and execution scope . . . 38
7.2 Business-rule, validation and integrity evaluation . . . 38

7.3 Usability, acceptance and feedback evidence 39

7.4 Professional judgment, ethics and accountable review 39

7.5 Team, communication and individual evidence 40

Chapter 8 - Conclusion 41

8.1 Implemented capabilities and evidence 41

8.2 Limitations and recommendations 41

8.3 References and evidence register 41

Chapter 1 - Introduction

1.1 Project overview

The Campus Equipment Borrowing and Inventory System is a Django web application for managing shared campus equipment from catalog browsing through request review, physical-unit reservation, release, returns and maintenance. It distinguishes a catalog item, such as a camera model, from each individually tagged physical unit. Students and other authorized borrowers use the Basic User portal; designated personnel use a custom Admin portal. [R1, pp.1-3]

This edition documents the actual source snapshot copied on 25 October 2026. The application was served from the separate IT401WProject folder with a fictional database outside the package. It is not a redesign or a new implementation. Existing development files, private configuration and the working database were preserved. Source inspection, migrated schema, recorded automated test results and actual browser captures support the statements below.

1.2 Background and problem statement

Campus equipment is shared across classes and activities. A useful control system must answer both how many items are currently borrowable and which exact units are reserved, issued, returned, damaged or retired. A request count alone cannot establish physical availability. The project specification therefore requires unit-level inventory, attributable decisions and a history that survives account or catalog changes. This is a requirements-derived problem analysis; no stakeholder interview or measured institutional baseline is claimed. [R1, pp.1-4]

The principal problems addressed are competing promises of the same unit, incomplete multi-item approval, unauthorized access to another borrower's records, inconsistent partial-return totals, and missing accountability for inventory changes. The system also needs a clear distinction between Pending interest and Approved reservations: Pending requests must not consume stock.

1.3 Objectives

The general objective is to provide an auditable, role-controlled borrowing lifecycle whose available stock and outstanding loans agree with physical-unit records.

1. Maintain categorized catalog records and unique physical-unit identities, condition, acquisition date and effective inventory status.

2. Accept one to three distinct requested items with positive, policy-limited quantities, a purpose and a valid desired return date.

3. Approve all requested units atomically, prevent concurrent double allocation, and release reservations when an Approved request is cancelled.

4. Record one loan per issued request and per-unit good or damaged returns, including partial returns and maintenance routing.

5. Calculate overdue status using Asia/Manila calendar dates and outstanding units, with no payment or fine subsystem.

6. Provide live dashboards, related-model reports, CSV exports, searchable audit history, setup instructions and reproducible tests.

1.4 Scope, significance and limitations

The scope includes the two campus roles, personal profiles, account administration, categories, catalog/physical units, borrowing decisions, release/slips, returns, maintenance, reporting, audit and borrowing settings. Borrowers gain a consistent view of serviceable and available quantities and their own history. Admin personnel gain exact asset accountability. Instructors receive runnable source, traceable evidence and a repeatable fictional demonstration.

The verified runtime is local Windows/Python 3.13.7/Django 6.1.2/SQLite. Paid hosting, email notifications, procurement, suppliers, fines, payments and external integrations are outside the implemented scope. No production-load, PostgreSQL, penetration-test, formal WCAG or legal-compliance certification is asserted. Desktop screenshots are verified; the complete mobile/assistive-technology evaluation remains unexecuted. Actual acceptance studies, team contributions, instructor policy approval and signatures remain external evidence gaps.

1.5 Options and recommendation

Option	Benefits and costs	Assessment
A. Server-rendered Django with custom portals	One validated server contract, ORM transactions, reusable templates and ordinary form fallback. Smaller dependency and deployment surface. Complex interactive flows need lightweight JavaScript.	Selected implementation; fits the required framework, two-role local demonstration and current maintainability constraints.
B. Django API with a separate single-page frontend	Independent client composition and richer client state, but additional API authorization/CSRF design, build tooling, deployment and duplicated validation effort.	Technically possible, but adds integration cost without a mandatory PDF need. Not implemented or experimentally compared.

The recommendation is to retain the current server-rendered design and improve it through small, tested components. This comparison is an engineering assessment against the specification, not a fabricated user preference survey or empirical performance benchmark. [R1, pp.4-5]

Chapter 2 - System Requirements

2.1 Source and traceability conventions

The authoritative source is the five-page IT 401W Equipment Borrowing specification for the first semester of AY 2026-2027. A1-A8, U1-U4 and BR1-BR6 are its original identifiers. Decimal sub-identifiers and RA/UI/DB/TECH/DEMO/SUB/EVID locators in the companion Test Cases PDF decompose mandatory prose for traceability; they are not additional instructor-assigned requirements. The registry contains 80 normalized checks and 116 scenarios. [R1]

Module	Mandatory behavior and actual implementation
A1 / accounts	Search users; inspect history; activate/deactivate and authorized role management without deleting historical transactions. Inactive accounts cannot sign in or keep transacting.
A2 / inventory	Create/edit/archive/restore categories and catalog items. Unique normalized asset identifiers, separate condition/acquisition fields, combined category/item/status/search filters and protected relationships.
A3 / decisions	Searchable paginated queue with status and date filters; current availability and purpose review; decision reason; atomic allocation on approval.
A4 / release	Approved request becomes one loan with issued actor/time, exact assets and agreed due date; printable borrowing slip; invalid or duplicate release refused.
A5 / returns	Return selected outstanding units, recording date/condition/notes/receiving Admin; partial/final return reconciliation; damaged returns create maintenance.
A6 / maintenance	Reported -> In progress -> Repaired or Retired, with notes and attributable events. Busy Reserved/Issued units cannot enter manual maintenance.
A7 / analytics	Available/Pending/Active/Overdue cards, monthly issued loans and category usage, date-filtered reports and CSV, all from ORM data.
A8 / control	Searchable attributable audit events and validated current quantity/duration policy. Existing agreed/issued terms are retained.
U1-U4 / Basic User	Register/sign in/sign out; own personal profile; active equipment search and counts; one-to-three-item requests; own tracking/history and eligible cancellation.

2.2 Roles and permissions

Operation	Basic User	Admin
Register / personal profile	Public registration creates BASIC_USER; own permitted personal fields only.	Sign in through same login; own profile. First Admin provisioned through documented setup.
Browse catalog	Active equipment, description, serviceable and available counts.	Internal catalog, units, maintenance and history.
Submit / track / cancel	Own requests only; cancel Pending/Approved before issue.	Review and manage authorized request lifecycle.
Approve / revise / issue / receive	Denied, including forged identifiers and staff flags.	Validated custom views/services, POST/CSRF and required reasons/confirmation.
Accounts / reports / audit / settings	Denied.	Admin role required; no generated Django Admin route.

Authorization uses ADMIN and BASIC_USER persisted roles, with current actor state rechecked in protected service operations. Django is_staff/is_superuser are framework fields, not a replacement for the campus role. A Basic User carrying those flags is still denied Admin endpoints. Owner-scoped querysets prevent direct-object-reference access to another borrower. [S1; T: role, owner and inactive-session tests]

2.3 Business rules

Rule	Enforced behavior	Verification boundary
BR1	Approval reserves the exact quantities atomically; failure rolls back all allocations. Approved cancellation releases reservations.	Atomicity, audit-failure rollback and cancel/issue concurrency assertions passed.
BR2	Each unit has one stored status: Available, Reserved, Issued, Maintenance or Retired. Non-Available units cannot be borrowed as available stock.	Inventory, allocation, release, return and maintenance transition tests passed.
BR3	Competing last-unit approvals have exactly one winner through transactions and active-allocation uniqueness.	Repeatable isolated SQLite concurrency tests passed; SQLite lacks effective row-level select_for_update locks.
BR4	One-to-three distinct items, positive quantities, every row validated before writes; no partial invalid request.	Service and forged HTTP cases passed, including manipulated legacy/settings limits.
BR5	Defaults: 3 units/item and 7 calendar days. Future/in-policy due date. Expired agreement cannot issue; Admin may revise Approved/unissued date with reason and audit.	Boundary, stale revision, rollback and revision/issue/cancel race tests passed.
BR6	Outstanding loans only; zero days before/on local due date; subsequent calendar-date difference.	Midnight, viewer timezone, partial/final return, dashboard/report/CSV consistency tests passed.

Policy detail
The hard distinct-item ceiling is three. Admin may lower max_items (1-3), set max_quantity (1-20), and set max_days (1-30). A revised approved due date is validated against the current policy measured from the original submission date. An expired date can be extended only while a valid future date still exists inside that policy window; the revision does not restart the borrowing window.

2.4 Non-functional and interface requirements

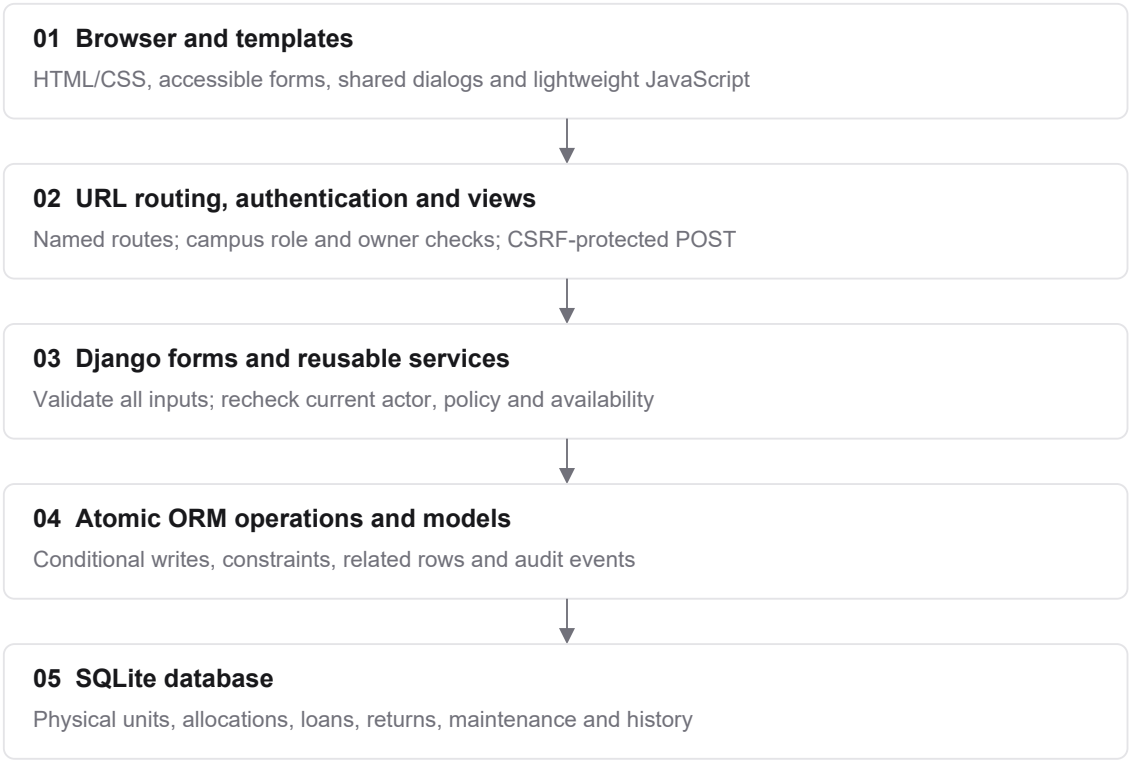
The application uses named Django routes, role-specific base templates, shared inputs/tables/badges/dialogs and a monochrome CSS token system. Form errors, textual status labels, empty states and confirmations are part of the interface. Horizontal table wrappers protect structured date/status values at small widths. Server validation remains authoritative even when JavaScript adds two-step selection or modal CRUD. [R1, pp.3-4]

Security requirements include CSRF-protected POST mutations, Django password hashing, escaped templates, owner checks, current active-role validation, private environment settings and exclusion of actual credentials from the package. Reliability relies on transaction boundaries, database constraints, current-availability rechecks and reproducible migration/seed setup. Accessibility and responsiveness are design obligations, not established certification from a desktop screenshot.

Chapter 3 - System Design

3.1 Django MTV architecture

Figure 3.1. Request processing and presentation



Response: views render inherited role templates; no separate frontend API is required.

The template layer presents state and contains no independent authoritative allocation logic. Views select owner/role-scoped records and bind existing forms. Reusable services validate current database state and coordinate transactions. Models declare relationships and constraints. Django ORM queries drive counts and reports; browser state cannot create an incomplete request while moving between steps. [S1-S7]

3.2 Technology stack and module boundaries

Technology / module	Purpose and justification
Python 3.13.7 / Django 6.1.2	Verified runtime. Django supplies auth, forms, server templates, ORM, migrations, CSRF and tests.
SQLite / Asia/Manila	Local demonstration database; date-aware calendar rules. File-level writer contention is a documented limitation.
asgiref 3.12.1 / sqlparse 0.6.0 / tzdata 2026.5	Pinned framework/runtime dependencies. No React, Tailwind or separate chart framework introduced.
HTML / CSS / JavaScript / SVG	Shared monochrome tokens, form/modal progressive enhancement and server-rendered chart graphics.
apps.accounts	Custom User, registration/sign-in/sign-out, profile, account activation/role services.
apps.inventory	Category, Equipment and EquipmentUnit; metadata, archives, aggregate inventory and eligibility.
apps.borrowing	Requests, items, active allocations; policy/date validation, decisions, revision and cancellation.
apps.loans	Loan and LoanUnit snapshots; atomic release, per-unit returns and calendar overdue properties.
apps.maintenance	Report, start, progress and resolve physical-unit maintenance.
apps.dashboard	Current policy, audit records, personal/Admin summaries, charts, reports and CSV.

3.3 Authentication and authorization design

Registration never offers Admin privilege and forged role/activation/permission fields do not grant it. The sign-in flow redirects authorized roles to /portal/ or /manage/. Django authentication hashes passwords and maintains server-side sessions; the session identifier cookie is not itself a record of campus permissions. Protected operations recheck account state so stale in-memory users or old sessions cannot restore deactivated access or overwrite a changed role.

Normal Admin work uses custom views, forms, templates and services. Django admin is installed as a framework component and its infrastructure table may exist after migration, but /admin/ is not mounted. Use the documented first-Admin setup, not the assumption that createsuperuser automatically grants the campus ADMIN role.

3.4 Inventory, reservation and loan design

Equipment is a catalog master. EquipmentUnit is one physical asset, with asset_tag, status, condition and acquisition_date. Counts are annotations/queries over units. Total Serviceable includes Available, Reserved and ON_LOAN (displayed Issued); Maintenance and Retired are excluded. Eligibility further excludes transaction-linked units even if a malformed legacy status were to say Available.

A Pending request stores desired catalog quantities only. Approval selects exact eligible units, creates Allocation rows and marks units Reserved inside one transaction. Allocation.active remains true through

issuance and becomes false per return or cancellation. Release creates one Loan and immutable issue snapshots in LoanUnit; later catalog renaming cannot rewrite the borrowing slip or historical report. Partial return leaves only unreturned LoanUnit rows outstanding.

3.5 Maintenance, audit and reporting design

Maintenance separates physical condition from inventory availability. A reported problem records its origin and Admin; start/complete actions are attributable. Successful repair restores Available and retirement keeps the unit unavailable. A damaged loan return creates the maintenance record atomically with its return, so a failed maintenance write cannot leave a half-recorded return.

AuditLog records actor and actor_username snapshot, action, entity type/identifier, occurred_at and a description. Due-date revisions include previous/new dates and reason in the event description; there is no invented dedicated revision table. Audit creation shares transaction boundaries with critical writes. Reports join loans, borrowers and unit snapshots and calculate outstanding/returned/overdue values. CSV uses the same filtered population and includes all matching rows, rather than only the displayed page.

Chapter 4 - Database Design

4.1 Actual schema and conceptual mapping

The migrated schema contains twelve domain models. The official PDF uses conceptual names that need not each correspond to an extra SQL table. The mapping below describes the real implementation. This section was generated from Django model metadata and checked against the included migrations; no additional business tables are proposed. [R1, p.3; S2-S7]

PDF concept	Actual model / storage
User / Profile	accounts.User extends AbstractUser and stores first/last name, email, contact_number, role and activation. No separate Profile table.
Category / CatalogItem	inventory.Category and inventory.Equipment.
EquipmentUnit	inventory.EquipmentUnit; status and physical condition are separate fields.
BorrowRequest / RequestItem	borrowing.BorrowingRequest and BorrowingRequestItem.
UnitAllocation	borrowing.Allocation links a request item to a physical unit. Tracking uses active; reservation time derives from request.approved_at. There is no Allocation.created_at field.
Loan / ReturnRecord	loans.Loan and LoanUnit. LoanUnit contains the issue snapshot and the per-unit returned_at, return_condition, return_notes and received_by fields; no separate ReturnRecord table.
MaintenanceRecord	maintenance.MaintenanceRecord, optionally linked to its damaged source_return.
AuditLog / SystemSetting	dashboard.AuditLog and dashboard.SystemSetting.

4.2 Rendered entity relationship diagrams

Each panel shows a domain slice; the following relationship register is authoritative for exact FK names, optionality and delete behavior. Connector labels in that register state source-to-target cardinality. Shared User and EquipmentUnit nodes recur for clarity.

Figure 4.1. Inventory and identity



Category 1 -> Equipment 0..N (nullable category_id); Equipment 1 -> EquipmentUnit 0..N. User is referenced by workflow actor/owner FKs.

Figure 4.2. Requests and allocations



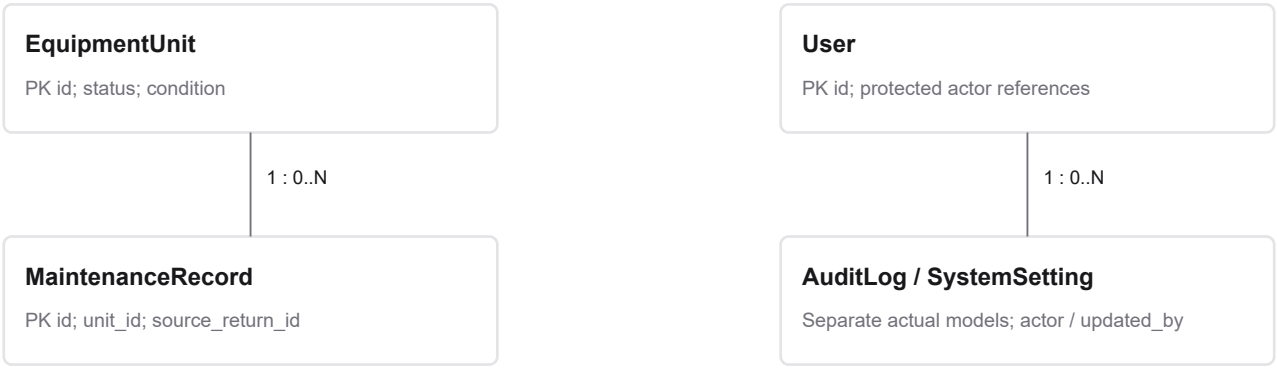
Request 1 -> Item 0..N; Item 1 -> Allocation 0..N; Unit 1 -> historical Allocation 0..N, with at most one active Allocation.

Figure 4.3. Issue and per-unit return history



Request 1 -> Loan 0..1; Loan 1 -> LoanUnit 0..N; Allocation 1 -> LoanUnit 0..1. Each LoanUnit also references one physical Unit.

Figure 4.4. Maintenance and accountability



Unit 1 -> MaintenanceRecord 0..N (one active repair); LoanUnit 1 -> MaintenanceRecord 0..1 via optional unique source_return. Audit references entity IDs without a domain FK.

4.3 Relationship register

Source field	Target	Relationship	Delete rule
accounts.User.groups	auth.Group	M:N through Django join table; optional	Framework M2M
accounts.User.user_permissions	auth.Permission	M:N through Django join table; optional	Framework M2M
inventory.Equipment.category	inventory.Category	0..N source rows : 1 target; optional	PROTECT
inventory.EquipmentUnit.equipment	inventory.Equipment	0..N source rows : 1 target; required	PROTECT
borrowing.BorrowingRequest.borrower	accounts.User	0..N source rows : 1 target; required	PROTECT
borrowing.BorrowingRequest.approved_by	accounts.User	0..N source rows : 1 target; optional	PROTECT
borrowing.BorrowingRequest.rejected_by	accounts.User	0..N source rows : 1 target; optional	PROTECT
borrowing.BorrowingRequest.cancelled_by	accounts.User	0..N source rows : 1 target; optional	PROTECT
borrowing.BorrowingRequestItem.request	borrowing.BorrowingRequest	0..N source rows : 1 target; required	CASCADE
borrowing.BorrowingRequestItem.equipment	inventory.Equipment	0..N source rows : 1 target; required	PROTECT
borrowing.Allocation.request_item	borrowing.BorrowingRequestItem	0..N source rows : 1 target; required	PROTECT
borrowing.Allocation.unit	inventory.EquipmentUnit	0..N source rows : 1 target; required	PROTECT
loans.Loan.request	borrowing.BorrowingRequest	0..1 source rows : 1 target; required	PROTECT
loans.Loan.borrower	accounts.User	0..N source rows : 1 target; required	PROTECT
loans.Loan.issued_by	accounts.User	0..N source rows : 1 target; required	PROTECT
loans.LoanUnit.loan	loans.Loan	0..N source rows : 1 target; required	PROTECT
loans.LoanUnit.allocation	borrowing.Allocation	0..1 source rows : 1 target; required	PROTECT
loans.LoanUnit.unit	inventory.EquipmentUnit	0..N source rows : 1 target; required	PROTECT

Source field	Target	Relationship	Delete rule
loans.LoanUnit.received_by	accounts.User	0..N source rows : 1 target; optional	PROTECT
maintenance.MaintenanceRecord.unit	inventory.EquipmentUnit	0..N source rows : 1 target; required	PROTECT
maintenance.MaintenanceRecord.responsible_admin	accounts.User	0..N source rows : 1 target; optional	PROTECT
maintenance.MaintenanceRecord.closed_by	accounts.User	0..N source rows : 1 target; optional	PROTECT
maintenance.MaintenanceRecord.source_return	loans.LoanUnit	0..1 source rows : 1 target; optional	PROTECT
dashboard.SystemSetting.updated_by	accounts.User	0..N source rows : 1 target; optional	PROTECT
dashboard.AuditLog.actor	accounts.User	0..N source rows : 1 target; required	PROTECT

4.4 Data dictionary conventions

All field names below are actual Django attributes; an FK SQL column normally appends _id, and the SQL column is shown when it differs. PK means primary key; UQ means field-level uniqueness. NULL describes database nullability; blank describes form/model validation allowance, which is distinct from NULL. M2M entries are relations stored through framework join tables rather than scalar columns in accounts_user. Auto-managed timestamps and service validation are explained separately from defaults.

4.5.1 accounts.User

SQL table: accounts_user. Custom authentication and embedded personal-profile identity. Role is constrained to ADMIN/BASIC_USER; ordinary profile editing cannot change privilege fields.

Field	Type	Keys / optionality	Meaning / relation / values
id	BigAutoField	PK; NULL no; blank yes	No explicit default/choices.
password	CharField(128)	NULL no; blank no	Encoded Django password hash; never disclose actual values
last_login	DateTimeField	NULL yes; blank yes	No explicit default/choices.
is_superuser	BooleanField	NULL no; blank no	Default: False
username	CharField(150)	UQ; NULL no; blank no	No explicit default/choices.
first_name	CharField(150)	NULL no; blank yes	No explicit default/choices.
last_name	CharField(150)	NULL no; blank yes	No explicit default/choices.
email	CharField(254)	NULL no; blank yes	No explicit default/choices.
is_staff	BooleanField	NULL no; blank no	Default: False
is_active	BooleanField	NULL no; blank no	Default: True
date_joined	DateTimeField	NULL no; blank no	Default: current timestamp (timezone.now)
role	CharField(10)	NULL no; blank no	Choices: BASIC_USER = Basic User, ADMIN = Admin; Default: BASIC_USER
contact_number	CharField(20)	NULL no; blank yes	No explicit default/choices.
groups	ManyToManyField	NULL no; blank yes; column None	-> auth.Group; M2M
user_permissions	ManyToManyField	NULL no; blank yes; column None	-> auth.Permission; M2M

Database constraints

accounts_valid_campus_role: CheckConstraint: condition=(AND: ('role__in', ['BASIC_USER', 'ADMIN'])) name='accounts_valid_campus_role'

4.5.2 inventory.Category

SQL table: inventory_category. Category master; is_active supports archive/restore. Case/space-normalized uniqueness includes archived names.

Field	Type	Keys / optionality	Meaning / relation / values
id	BigAutoField	PK; NULL no; blank yes	No explicit default/choices.
name	CharField(100)	NULL no; blank no	No explicit default/choices.
description	TextField	NULL no; blank yes	No explicit default/choices.
is_active	BooleanField	NULL no; blank no	Default: True

Database constraints

inventory_category_name_ci: UniqueConstraint: expressions=(Lower(Trim(F(name))),) name='inventory_category_name_ci'

Default ordering: name, pk.

4.5.3 inventory.Equipment

SQL table: inventory_equipment. Catalog metadata with an intentionally nullable legacy category FK. Normal Admin create/edit requires a valid eligible category. Archive is logical, not deletion.

Field	Type	Keys / optionality	Meaning / relation / values
id	BigAutoField	PK; NULL no; blank yes	No explicit default/choices.
name	CharField(150)	NULL no; blank no	No explicit default/choices.
description	TextField	NULL no; blank yes	No explicit default/choices.
category	ForeignKey	NULL yes; blank yes; column category_id	-> inventory.Category; PROTECT
is_active	BooleanField	NULL no; blank no	Default: True

Default ordering: name, pk.

4.5.4 inventory.EquipmentUnit

SQL table: inventory_equipmentunit. Physical asset. Known condition choices are independent of availability. Legacy unknown condition/date remain NULL; no fabricated acquisition dates are assigned.

Field	Type	Keys / optionality	Meaning / relation / values
id	BigAutoField	PK; NULL no; blank yes	No explicit default/choices.
equipment	ForeignKey	NULL no; blank no; column equipment_id	-> inventory.Equipment; PROTECT
asset_tag	CharField(80)	UQ; NULL no; blank no	No explicit default/choices.
status	CharField(12)	NULL no; blank no	Choices: AVAILABLE = Available, RESERVED = Reserved, ON_LOAN = On loan, MAINTENANCE = Maintenance, RETIRED = Retired; Default: AVAILABLE
condition	CharField(7)	NULL yes; blank yes	Choices: GOOD = Good, FAIR = Fair, POOR = Poor, DAMAGED = Damaged
acquisition_date	DateField	NULL yes; blank yes	No explicit default/choices.

Database constraints

inventory_valid_unit_status: CheckConstraint: condition=(AND: ('status__in', ['AVAILABLE', 'RESERVED', 'ON_LOAN', 'MAINTENANCE', 'RETIRED'])) name='inventory_valid_unit_status'

inventory_valid_unit_condition: CheckConstraint: condition=(OR: ('condition__isnull', True), ('condition__in', ['GOOD', 'FAIR', 'POOR', 'DAMAGED'])) name='inventory_valid_unit_condition'

inventory_unit_asset_tag_ci: UniqueConstraint: expressions=(Lower(Trim(F(asset_tag))),) name='inventory_unit_asset_tag_ci'

Default ordering: asset_tag.

4.5.5 borrowing.BorrowingRequest

SQL table: borrowing_borrowingrequest. Owner, purpose, agreed due date and decision/cancellation metadata. Partial-return and overdue labels are not separate stored request states.

Field	Type	Keys / optionality	Meaning / relation / values
id	BigAutoField	PK; NULL no; blank yes	No explicit default/choices.
borrower	ForeignKey	NULL no; blank no; column borrower_id	-> accounts.User; PROTECT
purpose	TextField(1000)	NULL no; blank no	No explicit default/choices.
requested_at	DateTimeField	NULL no; blank no	Default: current timestamp (timezone.now)
due_at	DateTimeField	NULL no; blank no	No explicit default/choices.
status	CharField(10)	NULL no; blank no	Choices: PENDING = Pending, APPROVED = Approved, REJECTED = Rejected, CANCELLED = Cancelled, ISSUED = Issued, COMPLETED = Completed; Default: PENDING
approved_at	DateTimeField	NULL yes; blank yes	No explicit default/choices.
approval_reason	TextField(300)	NULL no; blank yes	No explicit default/choices.
approved_by	ForeignKey	NULL yes; blank yes; column approved_by_id	-> accounts.User; PROTECT
rejection_reason	TextField(1000)	NULL no; blank yes	No explicit default/choices.
rejected_at	DateTimeField	NULL yes; blank yes	No explicit default/choices.
rejected_by	ForeignKey	NULL yes; blank yes; column rejected_by_id	-> accounts.User; PROTECT
cancelled_at	DateTimeField	NULL yes; blank yes	No explicit default/choices.
cancelled_by	ForeignKey	NULL yes; blank yes; column cancelled_by_id	-> accounts.User; PROTECT

Database constraints

borrowing_valid_request_status: CheckConstraint: condition=(AND: ('status__in', ['PENDING', 'APPROVED', 'REJECTED', 'CANCELLED', 'ISSUED', 'COMPLETED'])) name='borrowing_valid_request_status'

borrowing_due_after_request: CheckConstraint: condition=(AND: ('due_at__gt', F(requested_at))) name='borrowing_due_after_request'

Default ordering: -requested_at, -pk.

4.5.6 borrowing.BorrowingRequestItem

SQL table: borrowing_borrowingrequestitem. One catalog quantity per request. Positive quantities and unique (request,equipment) are database constrained.

Field	Type	Keys / optionality	Meaning / relation / values
id	BigAutoField	PK; NULL no; blank yes	No explicit default/choices.
request	ForeignKey	NULL no; blank no; column request_id	-> borrowing.BorrowingRequest; CASCADE
equipment	ForeignKey	NULL no; blank no; column equipment_id	-> inventory.Equipment; PROTECT
quantity	PositiveIntegerField	NULL no; blank no	No explicit default/choices.

Database constraints

borrowing_one_equipment_per_request: UniqueConstraint: fields=('request', 'equipment') name='borrowing_one_equipment_per_request'

borrowing_positive_quantity: CheckConstraint: condition=(AND: ('quantity__gt', 0)) name='borrowing_positive_quantity'

Default ordering: pk.

4.5.7 borrowing.Allocation

SQL table: borrowing_allocation. Physical-unit reservation link. Active persists through issue until per-unit return or cancellation. Approval timestamp is on the parent request.

Field	Type	Keys / optionality	Meaning / relation / values
id	BigAutoField	PK; NULL no; blank yes	No explicit default/choices.
request_item	ForeignKey	NULL no; blank no; column request_item_id	-> borrowing.BorrowingRequestItem; PROTECT
unit	ForeignKey	NULL no; blank no; column unit_id	-> inventory.EquipmentUnit; PROTECT
active	BooleanField	NULL no; blank no	Default: True

Database constraints

borrowing_unique_item_unit: UniqueConstraint: fields=('request_item', 'unit') name='borrowing_unique_item_unit'

borrowing_one_active_allocation: UniqueConstraint: fields=('unit',) name='borrowing_one_active_allocation' condition=(AND: ('active', True))

Default ordering: pk.

4.5.8 loans.Loan

SQL table: loans_loan. Exactly one issued loan per borrowing request; records issuing Admin, issue time, agreed due time and eventual completion.

Field	Type	Keys / optionality	Meaning / relation / values
id	BigAutoField	PK; NULL no; blank yes	No explicit default/choices.
request	OneToOneField	UQ; NULL no; blank no; column request_id	-> borrowing.BorrowingRequest; PROTECT
borrower	ForeignKey	NULL no; blank no; column borrower_id	-> accounts.User; PROTECT
issued_by	ForeignKey	NULL no; blank no; column issued_by_id	-> accounts.User; PROTECT
issued_at	DateTimeField	NULL no; blank no	Default: current timestamp (timezone.now)
due_at	DateTimeField	NULL no; blank no	No explicit default/choices.
completed_at	DateTimeField	NULL yes; blank yes	No explicit default/choices.

Database constraints

loan_due_after_issue: CheckConstraint: condition=(AND: ('due_at__gt', F(issued_at))) name='loan_due_after_issue'

loan_completion_after_issue: CheckConstraint: condition=(OR: ('completed_at__isnull', True), ('completed_at__gte', F(issued_at))) name='loan_completion_after_issue'

Explicit indexes: loan_borrower_completion (borrower, completed_at); loan_completion_due (completed_at, due_at).

Default ordering: -issued_at, -pk.

4.5.9 loans.LoanUnit

SQL table: loans_loanunit. One allocation issue plus immutable equipment/asset snapshots and the return record. Outstanding means returned_at is NULL.

Field	Type	Keys / optionality	Meaning / relation / values
id	BigAutoField	PK; NULL no; blank yes	No explicit default/choices.
loan	ForeignKey	NULL no; blank no; column loan_id	-> loans.Loan; PROTECT
allocation	OneToOneField	UQ; NULL no; blank no; column allocation_id	-> borrowing.Allocation; PROTECT
unit	ForeignKey	NULL no; blank no; column unit_id	-> inventory.EquipmentUnit; PROTECT
equipment_name	CharField(150)	NULL no; blank no	No explicit default/choices.
equipment_description	TextField	NULL no; blank yes	No explicit default/choices.
asset_tag	CharField(80)	NULL no; blank no	No explicit default/choices.
returned_at	DateTimeField	NULL yes; blank yes	No explicit default/choices.
return_condition	CharField(7)	NULL no; blank yes	Choices: GOOD = Good / undamaged, DAMAGED = Damaged
return_notes	TextField(1000)	NULL no; blank yes	No explicit default/choices.
received_by	ForeignKey	NULL yes; blank yes; column received_by_id	-> accounts.User; PROTECT

Database constraints

loan_one_active_issue_per_unit: UniqueConstraint: fields=('unit',) name='loan_one_active_issue_per_unit' condition=(AND: ('returned_at__isnull', True))

loan_complete_return_record: CheckConstraint: condition=(OR: (AND: ('received_by__isnull', True), ('return_condition', ''), ('return_notes', ''), ('returned_at__isnull', True)), (AND: ('received_by__isnull', False), ('return_condition__in', ['GOOD', 'DAMAGED']), ('returned_at__isnull', False))) name='loan_complete_return_record'

Default ordering: asset_tag, pk.

4.5.10 maintenance.MaintenanceRecord

SQL table: maintenance_maintenancerecord. Unit problem/progress/outcome with original reporting and closing actors. Start actor/time is available in audit history, not an invented started_at column.

Field	Type	Keys / optionality	Meaning / relation / values
id	BigAutoField	PK; NULL no; blank yes	No explicit default/choices.
unit	ForeignKey	NULL no; blank no; column unit_id	-> inventory.EquipmentUnit; PROTECT
reason	TextField(1000)	NULL no; blank no	No explicit default/choices.
status	CharField(12)	NULL no; blank no	Choices: REPORTED = Reported, IN_PROGRESS = In progress, REPAIRED = Repaired, RETIRED = Retired; Default: REPORTED
reported_at	DateTimeField	NULL no; blank no	Default: current timestamp (timezone.now)
completed_at	DateTimeField	NULL yes; blank yes	No explicit default/choices.
repair_notes	TextField(2000)	NULL no; blank yes	No explicit default/choices.
responsible_admin	ForeignKey	NULL yes; blank yes; column responsible_admin_id	-> accounts.User; PROTECT
closed_by	ForeignKey	NULL yes; blank yes; column closed_by_id	-> accounts.User; PROTECT
source_return	OneToOneField	UQ; NULL yes; blank yes; column source_return_id	-> loans.LoanUnit; PROTECT

Database constraints

maintenance_valid_status: CheckConstraint: condition=(AND: ('status__in', ['REPORTED', 'IN_PROGRESS', 'REPAIRED', 'RETIRED'])) name='maintenance_valid_status'

maintenance_one_active_per_unit: UniqueConstraint: fields=('unit',) name='maintenance_one_active_per_unit' condition=(AND: ('status__in', ('REPORTED', 'IN_PROGRESS')))

maintenance_coherent_resolution: CheckConstraint: condition=(OR: (AND: ('closed_by__isnull', True), ('completed_at__isnull', True), ('status__in', ('REPORTED', 'IN_PROGRESS'))), (AND: ('closed_by__isnull', False), ('completed_at__isnull', False), ('status__in', ['REPAIRED', 'RETIRED']), (NOT (AND: ('repair_notes', ''))))) name='maintenance_coherent_resolution'

maintenance_completion_after_report: CheckConstraint: condition=(OR: ('completed_at__isnull', True), ('completed_at__gte', F(reported_at))) name='maintenance_completion_after_report'

Default ordering: -reported_at, -pk.

4.5.11 dashboard.SystemSetting

SQL table: dashboard_systemsetting. Optional singleton row (id=1). Without a row, application defaults are 3 items/3 units/7 calendar days.

Field	Type	Keys / optionality	Meaning / relation / values
id	PositiveSmallIntegerField	PK; NULL no; blank no	Default: 1
max_items	PositiveSmallIntegerField	NULL no; blank no	Default: 3
max_quantity	PositiveSmallIntegerField	NULL no; blank no	Default: 3
max_days	PositiveSmallIntegerField	NULL no; blank no	Default: 7
updated_at	DateTimeField	NULL no; blank yes	Auto-updated on save
updated_by	ForeignKey	NULL yes; blank yes; column updated_by_id	-> accounts.User; PROTECT

Database constraints

dashboard_single_policy: CheckConstraint: condition=(AND: ('pk', 1)) name='dashboard_single_policy'

dashboard_valid_borrowing_limits: CheckConstraint: condition=(AND: ('max_days__gte', 1), ('max_days__lte', 30), ('max_items__gte', 1), ('max_items__lte', 3), ('max_quantity__gte', 1), ('max_quantity__lte', 20)) name='dashboard_valid_borrowing_limits'

4.5.12 dashboard.AuditLog

SQL table: dashboard_auditlog. Attributable event record with actor username snapshot and textual record reference. entity_type/entity_id are a logical reference, not a database FK.

Field	Type	Keys / optionality	Meaning / relation / values
id	BigAutoField	PK; NULL no; blank yes	No explicit default/choices.
actor	ForeignKey	NULL no; blank no; column actor_id	-> accounts.User; PROTECT
actor_username	CharField(150)	NULL no; blank no	No explicit default/choices.
action	CharField(32)	NULL no; blank no	Choices listed below this table.
entity_type	CharField(40)	NULL no; blank no	No explicit default/choices.
entity_id	PositiveBigIntegerField	NULL no; blank no	No explicit default/choices.
occurred_at	DateTimeField	NULL no; blank no	Default: current timestamp (timezone.now)
description	CharField(500)	NULL no; blank no	No explicit default/choices.

action choices: USER_ACTIVATED = User activated, USER_DEACTIVATED = User deactivated, USER_ROLE_CHANGED = User role changed, CATEGORY_CREATED = Category created, CATEGORY_UPDATED = Category updated, CATEGORY_DELETED = Category deleted, EQUIPMENT_CREATED = Equipment created, EQUIPMENT_UPDATED = Equipment updated, EQUIPMENT_ARCHIVED = Equipment archived, EQUIPMENT_RESTORED = Equipment restored, UNIT_ADDED = Physical unit added, UNIT_UPDATED = Physical unit updated, UNIT_RETIRED = Physical unit retired, REQUEST_APPROVED = Borrowing request approved, REQUEST_REJECTED = Borrowing request rejected, REQUEST_CANCELLED = Borrowing request cancelled, REQUEST_DUE_DATE_REVISED = Approved due date revised, LOAN_RELEASED = Equipment released, UNITS_RETURNED = Equipment returned, MAINTENANCE_REPORTED = Maintenance reported, MAINTENANCE_UPDATED = Maintenance progress updated, MAINTENANCE_REPAIRED = Maintenance completed: repaired, MAINTENANCE_RETIRED = Maintenance completed: retired, SETTINGS_UPDATED = Borrowing settings updated

Database constraints

dashboard_valid_audit_action: CheckConstraint: condition=(AND: ('action__in', ['USER_ACTIVATED', 'USER_DEACTIVATED', 'USER_ROLE_CHANGED', 'CATEGORY_CREATED', 'CATEGORY_UPDATED', 'CATEGORY_DELETED', 'EQUIPMENT_CREATED', 'EQUIPMENT_UPDATED', 'EQUIPMENT_ARCHIVED', 'EQUIPMENT_RESTORED', 'UNIT_ADDED', 'UNIT_UPDATED', 'UNIT_RETIRED', 'REQUEST_APPROVED', 'REQUEST_REJECTED', 'REQUEST_CANCELLED', 'REQUEST_DUE_DATE_REVISED', 'LOAN_RELEASED', 'UNITS_RETURNED', 'MAINTENANCE_REPORTED', 'MAINTENANCE_UPDATED', 'MAINTENANCE_REPAIRED', 'MAINTENANCE_RETIRED', 'SETTINGS_UPDATED'])) name='dashboard_valid_audit_action'

Explicit indexes: audit_entity_lookup (entity_type, entity_id).

Default ordering: -occurred_at, -pk.

4.6 Infrastructure, migrations and validation layers

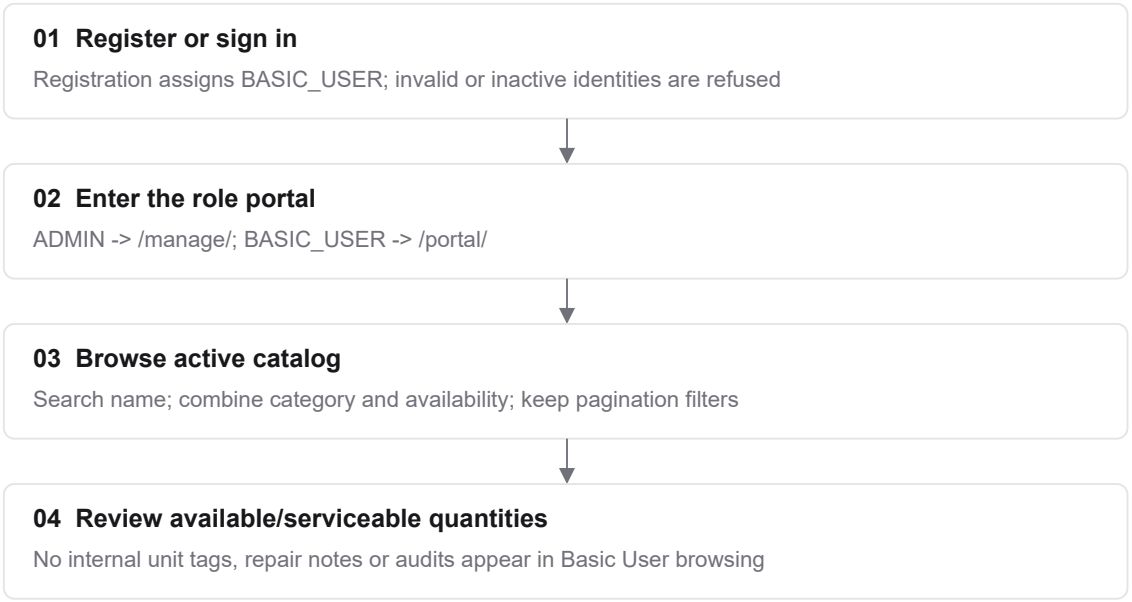
Django also creates auth_group, auth_permission, their joins, content types, sessions, migration history and django_admin_log infrastructure. The presence of django_admin_log does not mean the generated Admin UI is exposed. accounts_user_groups and accounts_user_user_permissions support AbstractUser permissions; campus role checks remain authoritative.

The submission includes 16 numbered domain migration files. Migration checks reported no model changes. The isolated database was migrated from scratch and the legacy migration regression test verified unknown condition/date preservation. Database constraints defend local invariants, while forms/services enforce cross-record role, policy, ownership, state-transition, current-availability and temporal rules. Direct ORM writes are not an authorized substitute for the application services.

Chapter 5 - System Workflows

5.1 Registration, sign-in and browsing

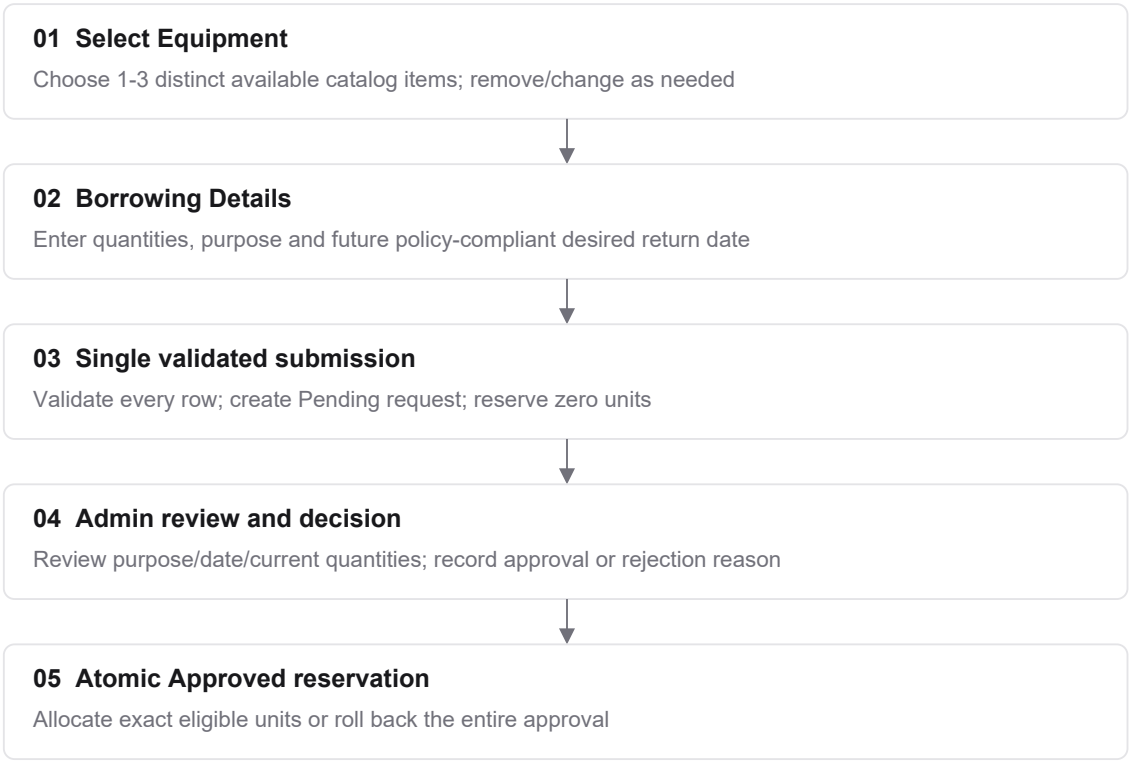
Figure 5.1. Authentication and equipment discovery



Clicking Request equipment carries the catalog identifier into the borrowing form. Valid available preselection appears in the selected summary; direct navigation has no preselected item. Invalid, archived, nonexistent or unavailable identifiers cannot create a selection. Users may add distinct equipment from any category, up to the hard ceiling of three.

5.2 Submission, review and allocation

Figure 5.2. Request to exact physical reservation



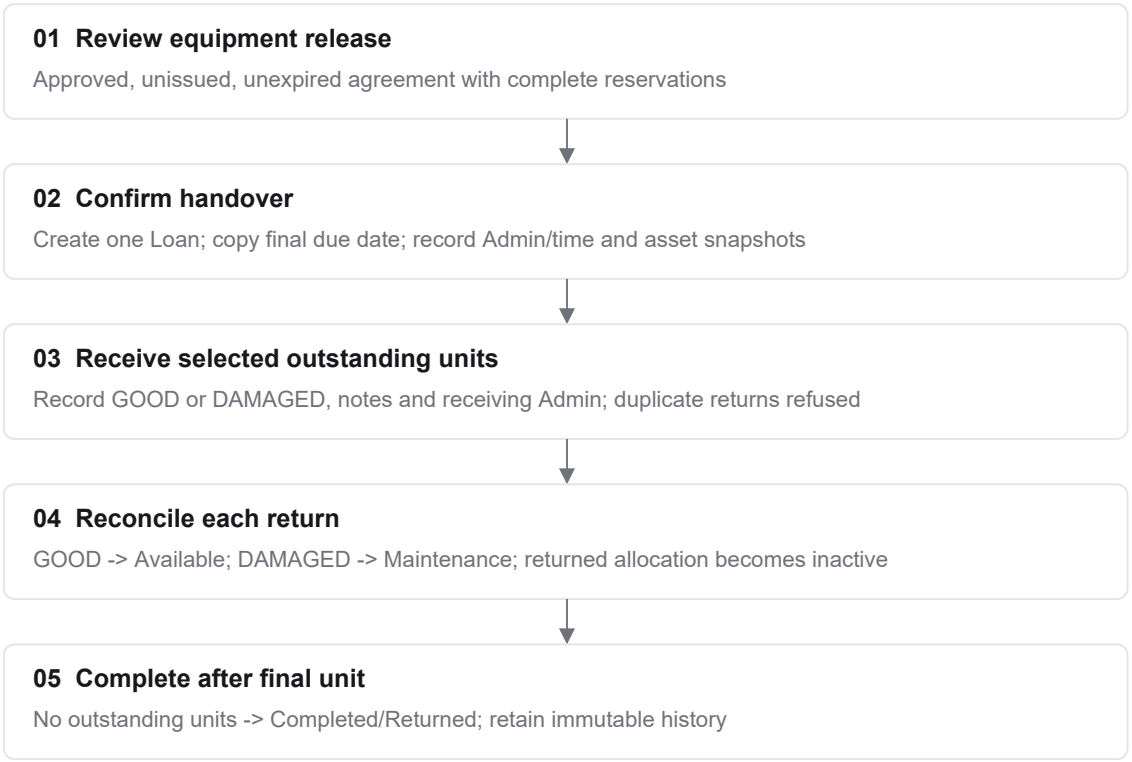
Branches: Reject -> Rejected; eligible cancellation -> Cancelled; Approved cancellation frees reservations.

The two UI steps share one final server submission; moving Back/Continue does not create draft database records. Quantity, purpose and date values are preserved when moving backward and on server validation errors. The enhanced interface never replaces backend checks. The no-JavaScript form remains a server-validated fallback.

The ordinary approval UI asks for a reason and allocates eligible assets automatically. The service can accept explicit unit mappings internally, but this documentation does not claim the Admin interface provides a manual asset-picker. Approval validates current availability again because multiple Pending requests can legitimately refer to the same free inventory.

5.3 Release, returns and overdue monitoring

Figure 5.3. Approved agreement through loan completion



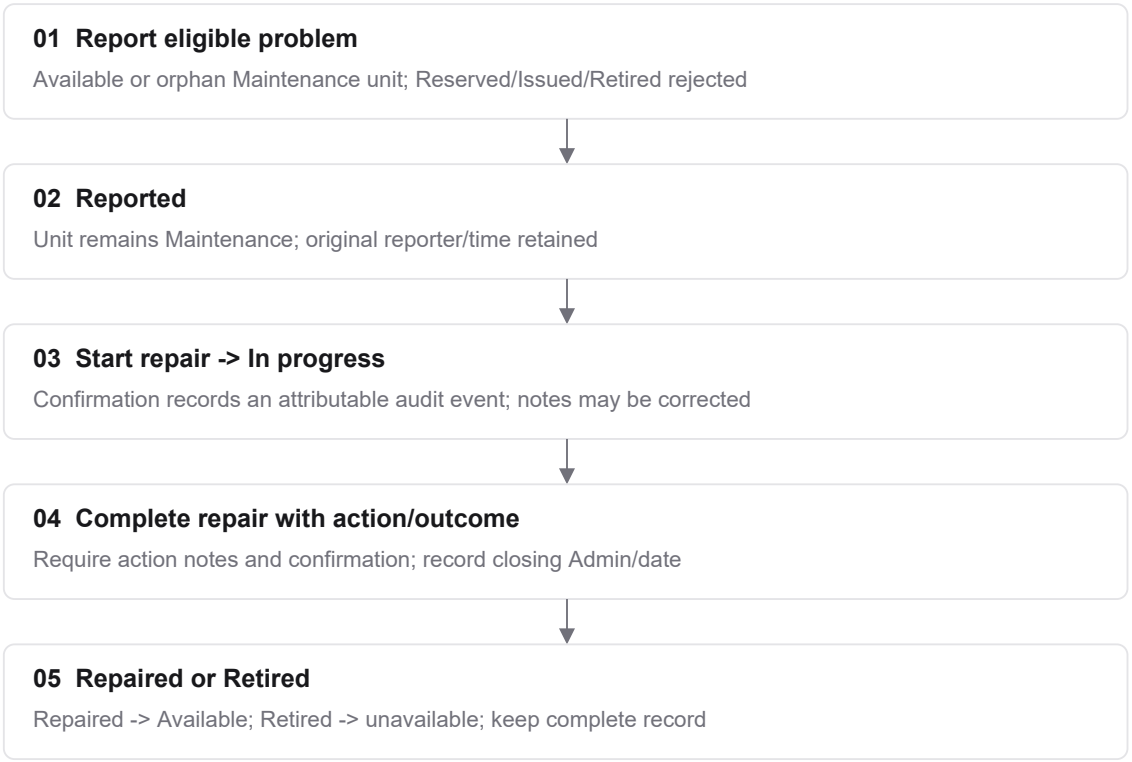
Before final return: outstanding loans become overdue only after the Asia/Manila due calendar date.

Admin may revise an Approved unissued due date with a required reason in a compact dialog. The service locks the agreement, checks an expected current date to reject stale edits, validates the new date against policy and records previous/new date, actor, time and reason in AuditLog. Issuance copies that final agreement. Revision cannot change an Issued/Completed loan; cancellation/issue races are tested.

For outstanding loans, overdue_days is max(0, Manila today minus Manila due date). A due time earlier on the same local day still yields zero overdue days. Crossing into the following local day yields one day even when fewer than 24 hours elapsed. Final return removes overdue classification. Issue expiration remains a timestamp check and is intentionally separate from calendar overdue reporting.

5.4 Maintenance and equipment archiving

Figure 5.4. Maintenance state and inventory outcome



Category and catalog archiving preserve relationships and historical requests. Archived categories remain visible to Admins but cannot be newly selected for unrelated/new equipment. Existing equipment may retain its archived category. Archiving a category does not implicitly archive its equipment. Catalog archiving is refused while live reservations/issues exist; an archived catalog item is hidden from Basic Users. Referenced deletion remains protected.

Chapter 6 - System Implementation

6.1 Project structure and reproducible setup

Path	Responsibility
apps/	Six domain apps with forms, services, models, views, URLs, management commands, tests and migrations.
config/	Django settings, environment parsing and root URL/WSGI/ASGI configuration.
templates/	base_admin.html, base_user.html, base_app.html, base_auth.html and shared components plus domain templates.
static/	Existing shared CSS variables, page styles, JavaScript enhancement and graphical assets.
scripts/demo.py	Existing safe fictional demo setup/verification/server helper; fresh local credentials are generated privately.
docs/	Four final PDFs and 26 actual screenshots; no obsolete audit drafts.
README.md / requirements.txt	Copy-specific setup, first Admin instructions, test commands, pinned dependencies and limitations.

Extract the ZIP, open its IT401WProject root, create a Python 3.13 virtual environment and install requirements.txt. Run python scripts/demo.py setup, verify and runserver for the isolated fictional demonstration. The helper creates local private .env.demo and demo.sqlite3 on the recipient machine. They are not included in this package. For an empty local database, follow README Option B, generate a new secret, migrate and create the first ADMIN role through the documented validated shell setup.

The ZIP is the authoritative snapshot. The existing public development repository is referenced in README, but cloning its latest published commit may differ from this copied working snapshot. No commit or push was performed during submission preparation. Avoid copying local .env, .env.demo, databases or generated credentials into a new archive.

6.2 Traceable implementation entry points

Area	Actual file, line and function
Accounts	apps/accounts/services.py:41 (set_user_active); apps/accounts/services.py:92 (update_profile)
Inventory eligibility	apps/inventory/services.py:24 (eligible_units); apps/inventory/services.py:32 (inventory_queryset)
Archives and units	apps/inventory/services.py:78 (archive_category); apps/inventory/services.py:165 (add_unit)
Request validation	apps/borrowing/services.py:31 (validate_due_date); apps/borrowing/services.py:60 (submit_request)
Atomic decision	apps/borrowing/services.py:98 (approve_request); apps/borrowing/services.py:221 (cancel_request)
Due-date revision	apps/borrowing/services.py:169 (revise_due_date)
Handover / returns	apps/loans/services.py:48 (release_request); apps/loans/services.py:99 (process_returns)
Overdue calendar	apps/loans/models.py:8 (local_day_start); apps/loans/models.py:52 (overdue_days)
Repair transitions	apps/maintenance/services.py:101 (start_repair); apps/maintenance/services.py:133 (resolve_record)
Reports / chart	apps/dashboard/services.py:71 (loan_report); apps/dashboard/services.py:102 (monthly_issued_loans)

6.3 Transaction and concurrency strategy

Critical services use atomic transactions, an initial conditional write/current-state recheck and select_for_update where supported. The partial unique active-allocation constraint prevents two active promises of one physical unit. Release/return constraints prevent duplicate active issue and repeated per-unit recording. Audit failures roll back their associated business transaction. Concurrency tests use separate connections and an isolated file-backed database, not the working development database.

SQLite serializes writers and select_for_update does not supply effective row locks. Contention may return a controlled conflict/locked response that requires a retry. The verified last-unit test produced one successful approval, not two. This is evidence for the current SQLite implementation, not proof of unlimited concurrent throughput or another database backend. Any database migration or deployment must repeat transaction, constraint and concurrency checks against that environment.

6.4 Maintenance, backup and operational guidance

1. Before schema upgrades, stop normal writes and create a restricted backup of the database and required private configuration. Never put those private backups in a public source ZIP.

2. Apply included migrations using the selected environment. Run check and makemigrations --check --dry-run; do not manually add missing columns or mark migrations as applied without verification.

3. Run the full test suite on an isolated database and reconcile demo/operational inventory before resuming service. Test restore procedures using a separate database.

4. Keep dependency versions and configuration changes recorded. Evaluate upgrades in an isolated environment before the live demonstration or deployment.

5. Use Archive/Restore and permitted lifecycle actions rather than deleting referenced records or directly editing stock totals. Investigate conflicts before retrying.

6. After a real deployment decision, assign owners for secret rotation, backups, access reviews, retention and incident response. No production deployment is included here.

6.5 Technology evaluation: optional barcode scanning

Barcode-assisted asset identification is a possible enhancement because the system already stores unique asset_tag values. A keyboard-wedge scanner could reduce manual transcription during Admin physical-unit lookup without replacing the existing validation/permission boundary. It must not automatically approve, release or return assets merely because a tag was scanned.

Criterion	Assessment and adoption gate
Effectiveness	Potentially faster exact-unit lookup; not measured. Compare scan accuracy and task time with current manual search using authorized participants before claiming improvement.
Compatibility	Keyboard input can reuse existing search fields, but device/browser behavior, duplicate input and tag formats need a proof of concept.
Security / privacy	Treat scanned data as untrusted input. Keep role/CSRF/confirmation checks; avoid embedding borrower data or secrets in labels.
User needs / cost	Useful where equipment volume justifies label printing and scanner support; the PDF does not require hardware procurement.
Recommendation	Defer adoption until a small supervised evaluation demonstrates benefit and maintenance ownership. No scanner integration or empirical trial is claimed.

Chapter 7 - Testing and Evaluation

7.1 Method and execution scope

The verification followed the source hierarchy: official requirements, actual code/schema, executed tests, then supporting documentation. Existing tests were run without changing application code to obtain a pass. A separate environment installed the pinned dependencies. The submission copy was migrated and seeded into an external fictional database and served on loopback for actual browser screenshots. The original tracked files and private database/configuration hashes were compared with their initial baseline.

Check	Observed result
Django suite	524 executed; 524 passed; 0 failed; 0 skipped. Test runner time: 63.894 seconds.
System checks	No issues (0 silenced).
Migration consistency	makemigrations --check --dry-run: No changes detected; fresh isolated migration succeeded.
Dependency check	pip check: no broken requirements.
Fictional seed	3 categories; 6 catalog items; 18 units; 1 Admin; 8 Basic Users; 20 requests; 8 loans; 5 maintenance records. All read-only reconciliation checks passed.
Runtime / screenshots	Copied app served on 127.0.0.1:8787; 26 actual screenshots at a consistent configured desktop viewport, including all 18 required filenames. PNG bounds: 1440 x 1061 (23) or 1425 x 1050 (3).
Original preservation	Source snapshot, private environment files and working database matched their initial hashes at verification.
Documentation cases	116 specification-derived scenarios. Status counts: PASS 108, NOT EXECUTED 3, BLOCKED 5. See companion Test Cases PDF for exact evidence and qualification.

The 524 automated methods and the 116 documented scenarios are different units of accounting. A scenario can cite several methods; one method may support several related clauses. A passing automated scenario means the cited assertions ran successfully, not that every manual procedure or every academic evidence obligation has been completed. The companion test register includes exact source files and method lines.

7.2 Business-rule, validation and integrity evaluation

Positive, negative and boundary assertions cover one/two/three item requests; fourth/duplicate/nonpositive/stale submissions; atomic multi-item reservation; mixed-status eligibility; duplicate/racing approvals, releases and returns; due-date revision authorization, policy boundaries, stale edits and rollback; Manila midnight overdue boundaries; partial/final returns; damaged return maintenance; archive restrictions; unique assets; legacy migrations; CSRF and cross-owner access.

BR1-BR6 have executed passing automated evidence in the current implementation. The last-unit race has one successful reservation. No application-level functional failure was observed in this run. This statement is limited to the tested assertions and environment; it does not erase the external evidence gaps or unexecuted physical-print, full responsive and assistive-technology evaluations.

7.3 Usability, acceptance and feedback evidence

Actual screenshots document the implemented hierarchy, filters, forms and navigation. They are visual evidence, not participant acceptance results. The supplied chat history records requests that influenced pre-existing work, such as clearer serviceable totals, compact summaries, modal quick actions and a two-step request selection flow. These messages are change context only; they are not signed institutional acceptance or quantified usability findings.

1. Before the live demonstration, agree observable acceptance criteria with authorized stakeholders: discover equipment, submit a valid request, reject invalid input, track own data and complete an Admin lifecycle.

2. Run tasks with appropriate Basic Users and Admins using fictional records. Record task outcome, error, assistance, observations and consented feedback without unnecessary personal data.

3. Log each issue with severity, requirement, responsible member, proposed action, verified fix and date. Retest the affected acceptance task and obtain actual sign-off where required.

4. Attach the genuine feedback, resolution and acceptance records to the instructor submission. Do not label a proposed evaluation as completed.

7.4 Professional judgment, ethics and accountable review

Issue / stakeholder	Safeguard and decision	Remaining accountable action
Privacy: borrowers and staff	Owner-scoped history; internal notes restricted; fictional screenshots and seed; no private data/credentials in ZIP.	Authorized custodian must approve retention, access and any real-data use under applicable institution policy.
Security: custodians and users	Hashed passwords, CSRF, POST-only writes, active-role checks, atomic services and audit.	Assign incident response/backups/access reviews; conduct security assessment before external exposure.
IP / attribution	Declare Django/runtime dependencies and AI assistance; inspect actual assets rather than claiming original authorship for all files.	Group must verify all contributor/third-party ownership and licensing, including any assets beyond known source.
Accessibility and fairness	Text status labels and visible errors; no color-only availability; predictable navigation. Allocation eligibility follows inventory/policy, not personal preference.	Test keyboard/screen-reader/mobile tasks with users; assess equitable access and remediate barriers.
Conflict / accountability	Admin decisions and due revisions require reasons and attributable audit events; retained records support review.	Agree an authorized appeal/correction process, role stewardship and supervision; do not delete contested history.
Public welfare / damaged assets	Damaged returns route to Maintenance; retired units stay unavailable.	Custodian must verify real repair safety before marking serviceable; software status is not a physical safety certificate.
Legal / codes / institution rules	This package identifies risks and required review, without asserting a legal or policy certification.	Instructor/authorized institutional reviewer must confirm applicable privacy, IP, AI and accessibility obligations and record actual decisions.

7.5 Team, communication and individual evidence

The official project calls for five to six members, approximately 75% Admin and 25% Basic User effort, individual contributions, rotated leadership, peer feedback, documented improvements and identifiable demonstration/Q&A segments. The actual section, verified roster and contribution records were not supplied. This package does not assign invented names, commits, leadership dates or work percentages to individuals. [R1, pp.1,5]

Required record	What the group must add
Allocation and contribution	Actual member/name/student identifier, assigned requirement, authored source/test/document evidence, dates and review references.
Rotated leadership	For every member: real leadership period, goal, assignment, coordination/progress evidence, obstacle and resolved action.
Peer/performance feedback	Dated feedback by/for each member, concrete improvement action and follow-up result.
Communication	Actual speaker segments, timed demonstration, Q&A participation, audience feedback and agreed actions.
Individual rubric	Evidence cross-references for PO-d, PO-e, PO-g, PO-h, PO-j and PO-n, using instructor interpretation rather than invented grades.

The user guide supplies a proposed twelve-minute agenda and three-minute Q&A. The technical README and this document serve developers/instructors; the illustrated guide serves Basic Users and Admins. Those artifacts support communication preparation, while actual delivery and feedback remain NOT EXECUTED. The separate AI Disclosure Agreement remains an unsigned draft until verified by proponents and reviewed against instructor/BulSU policy.

Chapter 8 - Conclusion

8.1 Implemented capabilities and evidence

The copied Django system supports the principal campus borrowing lifecycle with exact physical-unit tracking, atomic reservation, controlled issuance, partial/full returns, maintenance, reporting and attributable administration. The current automated suite passed all 524 methods. The package includes runnable source, sixteen domain migration files, pinned dependencies, a safe fictional seed/setup workflow, rendered technical/test/user documentation and actual system screenshots.

8.2 Limitations and recommendations

Technical verification supports the tested local implementation, including all six business rules. It does not establish complete academic submission compliance: externally held membership/contribution/acceptance evidence, signed disclosure and instructor credential delivery remain pending. Complete physical printing, full mobile/tablet and assistive-technology checks before making claims about those outcomes. Re-run the same suite and concurrency cases on any new environment.

Before instructor upload, verify section/proponent details, complete genuine team records and signatures, confirm AI/asset declarations and policy, privately deliver locally generated demonstration credentials, and rename a copy of the archive to IT401W_ProjectA_Section_Group.zip using actual values. The requested deliverable is IT401WProject-Final.zip; no section value has been invented.

8.3 References and evidence register

[R1] IT 401W - Project (Equipment Borrowing) - 1st Sem 26-27.pdf. Bulacan State University, five pages. A1-A8/U1-U4, logical model/workflow, BR1-BR6 and evidence/submission requirements. Source SHA-256: 77903c0959180782d2b94639d24e824155d0edf08ef65caa76222573a4502bff.

[S1] apps/accounts/models.py, services.py, forms.py, views.py and account tests; config/settings.py and config/urls.py in the submitted snapshot.

[S2] apps/inventory/models.py, services.py, forms.py, views.py and numbered migrations; actual Category, Equipment and EquipmentUnit schema.

[S3] apps/borrowing/models.py, services.py, forms.py, views.py and test_due_revisions.py; request/allocation/policy and revision implementation.

[S4] apps/loans/models.py, services.py, forms.py, views.py and test_calendar_overdue.py; loan snapshots, returns and calendar overdue behavior.

[S5] apps/maintenance/models.py, services.py, forms.py, views.py and tests.py; report/start/progress/outcome workflow.

[S6] apps/dashboard/models.py, services.py, views.py and tests.py; policy, audit, charts, related reports and CSV.

[S7] templates/, static/, scripts/demo.py and apps/inventory/management/commands/seed_project_demo.py; shared UI and fictional demonstration setup.

[T] Group 1 - Test Cases.pdf, execution IT401W-G1-20261025. Exact executed method references, requirement matrix and evidence-qualified scenario outcomes. Tests remain included as source for reproduction.

[B] docs/screenshots/, 26 actual views captured from the copied application at a configured 1440 x 1100 desktop viewport on 25 October 2026, using fictional data. The System User Guide identifies captions and contexts.